

Homework #1 – Solutions

1. Write BNF or EBNF description for “array” declaration in C language. You may assume that data types used for arrays are int, float, double, char, and their pointers; and initialization is not considered in your array definition.

Answer: For this question you can assume that array declarations start with a data type, followed by any number of array names and end with a semicolon. Array names start with an identifier followed by an integer constant between square brackets. Integer constant is either an integer value or an identifier. So the below grammar can be used to describe these syntactic rules.

$\langle \text{array_declaration} \rangle \rightarrow \langle \text{data_type} \rangle \langle \text{array_name} \rangle \{, \langle \text{array_name} \rangle\} ;$

$\langle \text{data_type} \rangle \rightarrow \text{int} \mid \text{float} \mid \text{double} \mid \text{char} \mid \text{int} * \mid \text{float} * \mid \text{double} * \mid \text{char} *$

$\langle \text{array_name} \rangle \rightarrow \langle \text{identifier} \rangle \langle \text{left_square_bracket} \rangle \langle \text{constant} \rangle \langle \text{right_square_bracket} \rangle$

$\langle \text{identifier} \rangle \rightarrow \langle \text{letter} \rangle \{ (\langle \text{letter} \rangle \mid \langle \text{digit} \rangle \mid _) \}$

$\langle \text{letter} \rangle \rightarrow \text{a} \mid \text{b} \mid \dots \mid \text{z} \mid \text{A} \mid \text{B} \mid \dots \mid \text{Z}$

$\langle \text{digit} \rangle \rightarrow 0 \mid 1 \mid 2 \mid \dots \mid 9$

$\langle \text{left_square_bracket} \rangle \rightarrow [$

$\langle \text{right_square_bracket} \rangle \rightarrow]$

$\langle \text{constant} \rangle \rightarrow \langle \text{digit} \rangle \{ \langle \text{digit} \rangle \} \mid \langle \text{identifier} \rangle$

2. Describe, in English, the language defined by the following grammar:

$\langle S \rangle \rightarrow \langle A \rangle \langle S \rangle \langle B \rangle \mid \epsilon$

$\langle A \rangle \rightarrow \text{a} \langle A \rangle \mid \text{a}$

$\langle B \rangle \rightarrow \text{b} \langle B \rangle \mid \text{b}$

For the above grammar ϵ means null.

Answer: To answer this question, we should do some derivations and then determine which strings are generated by the given grammar.

$\langle S \rangle \Rightarrow \langle A \rangle \langle S \rangle \langle B \rangle$

$\Rightarrow \text{a} \langle A \rangle \langle S \rangle \langle B \rangle$

$\Rightarrow \text{a a} \langle A \rangle \langle S \rangle \langle B \rangle$

$\Rightarrow a a a \langle S \rangle \langle B \rangle$

$\Rightarrow a a a \langle B \rangle$

$\Rightarrow a a a b$

The given grammar generates a null string or strings start with any number of **a**'s followed by any number of **b**'s. More formally, strings of the form **aⁿb^m** where $n \geq 0$ and $m \geq 0$ are generated.

3. Prove that the following grammar is ambiguous. After that re-write the below rules so that the new rules are unambiguous and $\wedge$ (means power) has precedence over $+$ operation.

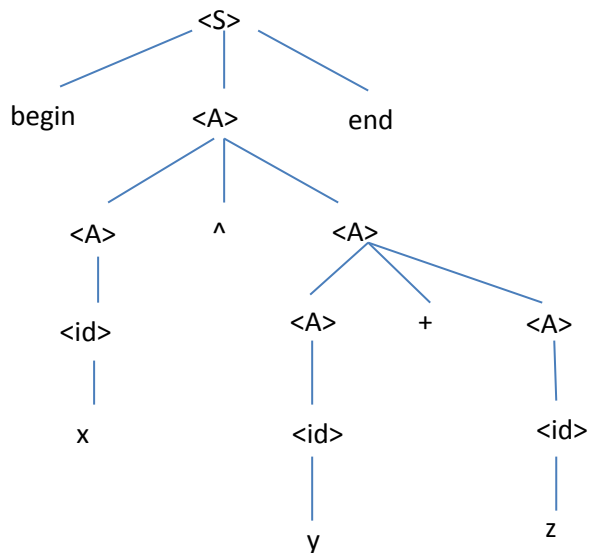
$\langle S \rangle \rightarrow \text{begin } \langle A \rangle \text{ end}$

$\langle A \rangle \rightarrow \langle A \rangle \wedge \langle A \rangle \mid \langle A \rangle + \langle A \rangle \mid \langle \text{id} \rangle$

$\langle \text{id} \rangle \rightarrow x \mid y \mid z$

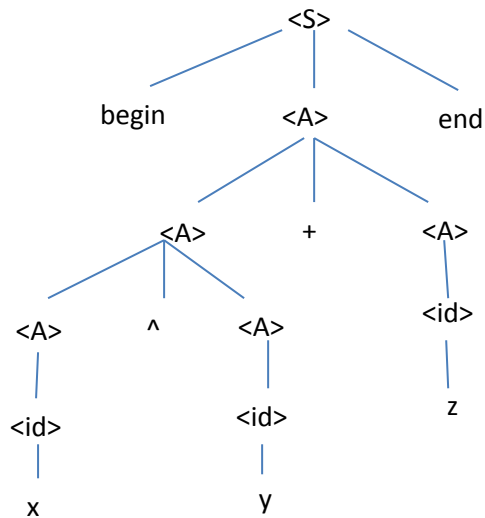
Answer: Choose a string for which two different parse trees can be drawn by using the grammar. If you can find such a string, then you prove that the grammar is ambiguous.

Let the string be **begin $x \wedge y + z$ end**



This parse tree first computes $y + z$ after that $\wedge$ is computed.

The second parse tree can be drawn as below:



In the second parse tree first $x \wedge y$ after that $+ z$ are computed.

The grammar should be changed as follows:

$\langle S \rangle \rightarrow \text{begin } \langle A \rangle \text{ end}$

$\langle A \rangle \rightarrow \langle B \rangle + \langle A \rangle \mid \langle B \rangle$

$\langle B \rangle \rightarrow \langle C \rangle \wedge \langle B \rangle \mid \langle C \rangle$

$\langle C \rangle \rightarrow \langle \text{id} \rangle$

$\langle \text{id} \rangle \rightarrow x \mid y \mid z$

By using the above grammar there is only one parse tree for the string **begin $x \wedge y + z$ end**

For all strings that are generated by this new grammar you can find only one parse tree, and $\wedge$ has precedence over $+$ operation, therefore the new grammar is unambiguous.

4. Prove that the below program is correct.

$\{ x > 0 \text{ OR } y > 0 \}$

if $(x > y)$

$b = x;$

else

$b = y;$

$\{b > 0\}$

Answer: To prove the above program, we start with the given post condition, and try to find the weakest precondition. If the weakest precondition that we will find is equal to the given precondition, then the program is correct.

Case I: $(x > y)$

$b = x; \{ b > 0 \}$

$\Rightarrow \{ x > 0 \}$

Case II: $(x \leq y)$

$b = y; \{ b > 0 \}$

$\Rightarrow \{ y > 0 \}$

The weakest precondition is the union of these two preconditions $\{ x > 0 \} \text{ OR } \{ y > 0 \}$. As the computed precondition is the same as the given precondition, the program is correct.